

**IZMIR UNIVERSITY OF ECONOMICS
FACULTY OF ENGINEERING
SOFTWARE ENGINEERING**

2025-2 SE410 Software Framework Applications



**Sustainable Product Shopping Platform
with Carbon Footprint Tracking
Version 2 Report**

Group #1

1. Introduction	3
Project Objectives	3
2. Requirements	3
Functional Requirements	3
Non-Functional Requirements	4
3. Database Schema	5
EF Core Configuration Highlights	6
4. LINQ Queries	6
LINQ-01: Filter Active Products — GetProducts()	6
LINQ-02: Cart Join with Products — GetCart()	7
LINQ-03: Eager Load Order Items — GetCustomerOrders()	7
LINQ-04: Category GroupBy — GetCatalogAnalytics()	7
LINQ-05: Revenue Per Merchant — GetCatalogAnalytics()	8
LINQ-06: Low-Stock Alerts — GetCatalogAnalytics()	8
LINQ-07: Monthly Carbon GroupBy — CarbonCalculator.BuildCustomerSummary()	9
5. Delegates and Events	9
Delegate Definitions — MarketplaceEvents.cs	9
Event Declarations — DatabaseMarketplaceService	10
Raising Events	10
Subscribing in Tests	10
6. MVC Structure	11
7. User Interface Design	11
Desktop Application — .NET MAUI (Merchant Console)	11
Web Application — Blazor Server (Customer Web)	12
REST API — ASP.NET Core Minimal API	12
8. Unit Tests	12
Running the Tests	14
9. Sources & How to Run	14
GitHub Repository	14
How to Run — Step by Step	15
1. Web API (ASP.NET Core)	15
2. Customer Web App (Blazor Server)	15
3. Merchant Desktop (MAUI)	15
4. Unit Tests (xUnit)	15
Video Links	15
10. Conclusion	16

1. Introduction

The Sustainable Product Shopping Platform (SustainablePS) is a marketplace application built with .NET 9 that allows merchants to list eco-friendly products and customers to browse, purchase, and track the carbon footprint of their orders.

Version 2 extends the original prototype with a production-grade SQLite database (relative path, zero-configuration), a new ASP.NET Core Web API, full delegate and event wiring, and a comprehensive xUnit test suite

Project Objectives

- Provide a dual-interface marketplace: a MAUI desktop console for merchants and a Blazor web app for customers.
- Track the carbon footprint (kg CO₂e) of every product and purchase.
- Expose a REST Web API that both interfaces share through a single SQLite database.
- Demonstrate C# language features: LINQ, delegates, events, EF Core, Minimal API.
- Include at least 15 xUnit unit tests covering all major scenarios.

2. Requirements

Functional Requirements

ID	Requirement	Description
FR-01	User Registration & Login	Users register as Customer or Merchant; PBKDF2 password hashing.
FR-02	Product Catalog	Merchants add, edit, delete products with name, price, stock, and CarbonKgPerUnit.
FR-03	Stock Management	Merchants adjust stock quantities; StockChanged event notifies subscribers.
FR-04	Shopping Cart	Customers add, remove, and adjust quantities; cart is persisted in SQLite.
FR-05	Checkout & Payment	MockPaymentProcessor validates card; atomic EF transaction updates stock.

FR-06	Order History	Customers view past orders; merchants view all orders containing their items.
FR-07	Order Status	Merchants advance order status (Confirmed → Preparing → Shipped → Delivered).
FR-08	Carbon Tracking	Per-order and lifetime carbon footprint displayed; monthly breakdown chart.
FR-09	Notifications	Order confirmation and shipping-update notifications sent to customers.
FR-10	Analytics (LINQ)	GetCatalogAnalytics() computes category stats, merchant revenue, low-stock alerts.
FR-11	REST API	ASP.NET Core Web API exposes all marketplace operations as JSON endpoints.

Non-Functional Requirements

ID	Requirement	Description
NFR-01	Portability	SQLite database uses a relative path so no configuration change is needed on any computer.
NFR-02	Security	Passwords hashed with PBKDF2-SHA256 + per-user salt; never stored as plaintext.
NFR-03	Reliability	Checkout uses an EF Core transaction; stock is never partially deducted.
NFR-04	Maintainability	XML doc comments on all public types; SOLID layer separation (Core / API / Web / MAUI).
NFR-05	Testability	DatabaseMarketplaceService accepts IDbContextFactory enabling in-memory DB in tests.
NFR-06	Performance	LINQ queries translated to SQL by EF Core; no over-fetching.

NFR-07	Scalability	Singleton service + factory pattern safely supports multiple concurrent requests.
NFR-08	Usability	Blazor web UI auto-populates demo credentials; MAUI merchant console requires no manual setup.

3. Database Schema

The application uses SQLite accessed through EF Core 9. The database file is resolved at runtime as:

```
var dbPath = Path.Combine(AppContext.BaseDirectory, "sustainableps.db");
```

Because `AppContext.BaseDirectory` is always the directory containing the compiled executable, the `.db` file travels with the application and requires no connection-string changes when moved between computers — satisfying the relative path requirement.

Table	Primary Key	Key Columns	Notes
Users	Id (GUID)	Email (UNIQUE), PasswordHash, Role, FullName	Role stored as string
Products	Id (GUID)	MerchantId (FK), Name, Price, StockQuantity, CarbonKgPerUnit, IsActive	Soft delete via IsActive
Orders	Id (GUID)	CustomerId (FK), TransactionId, Status, PaymentStatus, CreatedAt	Status stored as string
OrderItems	Id (GUID)	OrderId (FK), ProductId, MerchantId, ProductName, UnitPrice, Quantity, CarbonKgPerUnit	CASCADE delete with Order
Notifications	Id (GUID)	UserId (FK), Type, Title, Message, IsRead, CreatedAt	Type stored as string

Table	Primary Key	Key Columns	Notes
CartEntries	Id (GUID)	CustomerId, ProductId, Quantity	UNIQUE(CustomerId, ProductId)

EF Core Configuration Highlights

Key mappings in MarketplaceDbContext.OnModelCreating:

```
entity.Property(u => u.Role).HasConversion<string>(); // enum → string
entity.HasIndex(u => u.Email).IsUnique();
entity.Property(p => p.Price).HasColumnType("TEXT"); // SQLite DECIMAL
entity.HasMany(o => o.Items) // 1-to-many
    .WithOne()
    .HasForeignKey(i => i.OrderId)
    .OnDelete(DeleteBehavior.Cascade);
entity.Ignore(o => o.TotalPrice); // computed, not stored
entity.HasIndex(c => new { c.CustomerId, c.ProductId }).IsUnique();
```

4. LINQ Queries

All LINQ queries use method syntax and are located in DatabaseMarketplaceService.cs. EF Core translates IQueryable chains to SQL; in-memory post-processing is used only where multi-set joins are clearer.

LINQ-01: Filter Active Products — GetProducts()

```
db.Products
    .Where(p => p.IsActive)
    .OrderBy(p => p.Name)
    .ToList();
```

Explanation: Translated to SQL: SELECT * FROM Products WHERE IsActive=1 ORDER BY Name. Used on every page load to populate the catalog.

LINQ-02: Cart Join with Products — GetCart()

```
db.CartEntries
    .Where(c => c.CustomerId == customerId)
    .Join(db.Products.Where(p => p.IsActive),
        c => c.ProductId,
        p => p.Id,
        (c, p) => new CartLine(p, c.Quantity))
    .OrderBy(line => line.Product.Name)
    .ToList();
```

Explanation: Join two tables (CartEntries x Products) to build CartLine projections. Filters soft-deleted products from cart display.

LINQ-03: Eager Load Order Items — GetCustomerOrders()

```
db.Orders
    .Include(o => o.Items)
    .Where(o => o.CustomerId == customerId)
    .OrderByDescending(o => o.CreatedAt)
    .ToList();
```

Explanation: Include generates a LEFT JOIN to fetch related OrderItems in one query, avoiding the N+1 problem.

LINQ-04: Category GroupBy — GetCatalogAnalytics()

```
activeProducts
    .GroupBy(p => p.Category)
    .Select(g => new CategoryStat(
        g.Key,
        g.Count(),
        Math.Round(g.Average(p => p.CarbonKgPerUnit), 2),
        g.Sum(p => p.StockQuantity)))
    .OrderBy(s => s.Category)
    .ToList();
```

Explanation: Groups products by category and projects aggregates (Count, Average, Sum) into CategoryStat records.

LINQ-05: Revenue Per Merchant — GetCatalogAnalytics()

```
paidOrders
    .SelectMany(o => o.Items)
    .GroupBy(item => item.MerchantId)
    .Select(g => new MerchantRevenueStat(
        MerchantId: g.Key,
        MerchantName: users.Where(u => u.Id == g.Key)
            .Select(u => u.FullName)
            .FirstOrDefault() ?? "Unknown",
        TotalRevenue: g.Sum(item => item.LineTotal),
        TotalCarbonKg: Math.Round(g.Sum(item => item.LineCarbonKg), 2),
        ItemsSold: g.Sum(item => item.Quantity)))
    .OrderByDescending(s => s.TotalRevenue)
    .ToList();
```

Explanation: SelectMany flattens order items; GroupBy merchant; FirstOrDefault resolves merchant name. Demonstrates projection into named record types.

LINQ-06: Low-Stock Alerts — GetCatalogAnalytics()

```
activeProducts
    .Where(p => p.StockQuantity <= LowStockThreshold) // const int = 5
    .OrderBy(p => p.StockQuantity)
    .Select(p => new LowStockAlert(p.Id, p.Name, p.StockQuantity))
    .ToList();
```

Explanation: Simple filter + projection. Result powers a low-stock warning panel in the merchant console.

LINQ-07: Monthly Carbon GroupBy — CarbonCalculator.BuildCustomerSummary()

```
paidOrders
    .GroupBy(o => new { o.CreatedAt.Year, o.CreatedAt.Month })
    .OrderBy(g => g.Key.Year).ThenBy(g => g.Key.Month)
    .Select(g => new MonthlyCarbonPoint(
        g.Key.Year, g.Key.Month,
        Round(g.Sum(o => o.TotalCarbonKg))))
    .ToList();
```

Explanation: Groups orders by Year+Month anonymous type, then projects into a timeline of MonthlyCarbonPoint records for the carbon dashboard chart.

5. Delegates and Events

Custom delegate types are defined in MarketplaceEvents.cs and exposed as events on DatabaseMarketplaceService. This decouples the business layer from UI layers: the service raises events, and any number of subscribers (MAUI pages, Blazor components, loggers) can attach handlers without the service knowing about them.

Delegate Definitions — MarketplaceEvents.cs

```
// Raised when an order is confirmed after a successful checkout.
public delegate void OrderConfirmedHandler(
    Guid customerId, Guid orderId, double totalCarbonKg);

// Raised when a product's stock quantity changes.
public delegate void StockChangedHandler(
    Guid productId, string productName, int newStock);

// Raised when a merchant advances an order's fulfillment status.
public delegate void OrderStatusChangedHandler(
    Guid orderId, string newStatus);

// Raised when a merchant adds a new product to the catalog.
```

```
public delegate void ProductAddedHandler(  
    Guid merchantId, string productName, double carbonKgPerUnit);
```

Event Declarations — DatabaseMarketplaceService

```
public event OrderConfirmedHandler? OrderConfirmed;  
public event StockChangedHandler? StockChanged;  
public event OrderStatusChangedHandler? OrderStatusChanged;  
public event ProductAddedHandler? ProductAdded;
```

Raising Events

Each event is raised with `?.Invoke()` after the database write completes, outside any transaction, to avoid re-entrancy issues:

```
// Inside Checkout() — after tx.Commit():  
OrderConfirmed?.Invoke(customerId, order.Id, order.TotalCarbonKg);  
  
// Inside AddProduct() — after db.SaveChanges():  
ProductAdded?.Invoke(merchantId, product.Name, product.CarbonKgPerUnit);  
  
// Inside UpdateStock() — after db.SaveChanges():  
StockChanged?.Invoke(product.Id, product.Name, product.StockQuantity);  
  
// Inside UpdateOrderStatus() — after db.SaveChanges():  
OrderStatusChanged?.Invoke(order.Id, status.ToString());
```

Subscribing in Tests

```
// Lambda assigned to the OrderConfirmedHandler delegate event:  
_svc.OrderConfirmed += (customerId, orderId, carbonKg) =>  
{  
    eventFired = true;  
    Console.WriteLine($"Order {orderId} confirmed — {carbonKg:F2} kg CO2e");  
};
```

6. MVC Structure

SustainablePS follows a layered architecture that maps to the Model-View-Controller pattern across its three applications:

Layer	Role	Components
Model	Domain entities & business logic	SustainablePS.Core: Product, Order, UserAccount, CartEntry, Notification, CartLine, DatabaseMarketplaceService, CarbonCalculator, MarketplaceDbContext
View	User interface rendering	SustainablePS.Web: Home.razor (Blazor)SustainablePS.Maui: MerchantPage.xaml, CatalogPage.xaml, CartPage.xaml, DashboardPage.xaml
Controller	Request handling & routing	SustainablePS.Api: Minimal API endpoints in Program.csSustainablePS.Web: @code blocks in Razor componentsSustainablePS.Maui: Event handlers in .xaml.cs files

Design decisions: The Core project is a class library with no UI dependency — it can be referenced by any host (MAUI, Blazor, API, test project). All three host projects register DatabaseMarketplaceService as a singleton, pointing at the same SQLite file via the relative path. This means the merchant desktop and the customer web UI share live data automatically.

7. User Interface Design

Desktop Application — .NET MAUI (Merchant Console)

The MAUI application targets macOS and Windows. It provides the merchant management console with four tab pages: MerchantPage (product management and order fulfillment), CatalogPage (product browsing), CartPage (demo cart), and DashboardPage (carbon summary).

- Login panel — auto-filled with demo merchant credentials.

- Product editor — add/edit name, category, description, price, stock, carbon per unit.
- Quick stock adjust — +/- buttons on each product row.
- Order list — advance status with one button click.
- Notification list — mark all read button.

Web Application — Blazor Server (Customer Web)

The Blazor Server application provides the customer shopping experience served at <http://localhost:5000> by default:

- Account panel — login, register (customer or merchant role), logout.
- Metric strip — total cart value, cart carbon, lifetime carbon, unread notifications.
- Catalog toolbar — real-time search input + category dropdown filter.
- Product list — add-to-cart buttons, carbon footprint per unit.
- Cart sidebar — quantity controls (+/-/Remove), carbon total, Pay button.
- Notification panel — order confirmation and shipping-update messages.
- Carbon dashboard — monthly CO2e chart and lifetime totals.
- Purchase history — order date, status, item count, total price, carbon.

REST API — ASP.NET Core Minimal API

The API project runs at <http://localhost:5001> and exposes all marketplace operations as JSON endpoints. OpenAPI/Swagger is enabled in development at </openapi/v1.json>:

- POST </api/auth/login>, POST </api/auth/register>
- GET </api/products>, POST </api/products>, PUT </api/products/{id}>, PATCH </api/products/{id}/stock>, DELETE </api/products/{id}>
- GET </api/cart/{customerId}>, POST </api/cart/add>, POST </api/cart/checkout>
- GET </api/orders/customer/{id}>, GET </api/orders/merchant/{id}>, PATCH </api/orders/{id}/status>
- GET </api/analytics>, GET </api/carbon/{customerId}>, GET </api/impact>
- GET </api/notifications/{userId}>, POST </api/notifications/{userId}/read-all>

8. Unit Tests

The test project `SustainablePS.Tests` uses xUnit with the EF Core InMemory provider. Each test gets a fresh isolated database (unique GUID name) so tests are independent and parallelisable.

ID	Test Method	What is Tested	Expected Result
TC-01	Login_ValidCredentials_ReturnsUser	Login with correct email+password	UserAccount returned with correct email and role
TC-02	Login_WrongPassword_Throws	Login with wrong password	InvalidOperationException on containing 'Invalid'
TC-03	Register_NewAccount_CanLogin	Register a new Customer account then login	Returned account ID matches login result
TC-04	Register_DuplicateEmail_Throws	Register with existing email	InvalidOperationException thrown
TC-05	GetProducts_ReturnsOnlyActiveOrderedByName	Product list after adding a product	All items IsActive=true; names in ascending order
TC-06	AddProduct_RaisesProductAddedEvent	Add product fires ProductAdded delegate event	capturedName == 'Hemp Soap'
TC-07	UpdateStock_RaisesStockChangedEvent	UpdateStock fires StockChanged delegate event	capturedStock == 99
TC-08	AddToCart_AddsProductToCart	AddToCart persists cart entry	Cart has 1 line with Quantity=2
TC-09	GetCartCarbon_CalculatesCorrectly	Carbon calculation for 3 units	carbon == round(3 x CarbonKgPerUnit, 2)
TC-10	Checkout_Success_CreatesOrderAndReducesStock	Successful checkout with demo card	result.Success=true; stock reduced by 2

ID	Test Method	What is Tested	Expected Result
TC-11	Checkout_DeclinedCard_StockUnchanged	Checkout with declined card	result.Success=false; stock unchanged
TC-12	Checkout_Success_RaisesOrderConfirmedEvent	OrderConfirmed event after checkout	eventFired == true
TC-13	UpdateOrderStatus_RaisesOrderStatusChangedEvent	OrderStatusChanged event when status advanced	capturedStatus == 'Shipped'
TC-14	GetCatalogAnalytics_ReturnsCategoryStats	LINQ analytics with seeded data	CategoryStats non-empty; ProductCount > 0
TC-15	GetCustomerCarbonSummary_IncludesSeededOrder	Carbon summary includes seeded demo order	OrderCount >= 1; TotalCarbonKg > 0

Running the Tests

```
# Run all xUnit tests:
cd tests/SustainablePS.Tests
dotnet test

# Expected output:
Passed! - Failed: 0, Passed: 15, Skipped: 0, Total: 15
```

All 15 tests are green. The InMemory provider means no real SQLite file is created during testing — tests run in milliseconds.

9. Sources & How to Run

GitHub Repository

<https://github.com/serhatuzunbayir/SustainableProductSPRING2025-26.git>

How to Run — Step by Step

Prerequisites: .NET 9 SDK (<https://dot.net/download>)

1. Web API (ASP.NET Core)

```
cd src/SustainablePS.Api
dotnet run
# Runs at: http://localhost:5001
# Swagger UI: http://localhost:5001/openapi/v1.json
```

2. Customer Web App (Blazor Server)

```
cd src/SustainablePS.Web
dotnet run
# Runs at: http://localhost:5000
# Login: customer@sustainable.test / demo123
```

3. Merchant Desktop (MAUI)

```
cd src/SustainablePS.Maui
dotnet build -t:Run -f net9.0-maccatalyst # macOS
dotnet build -t:Run -f net9.0-windows10.0.19041 # Windows
# Login: merchant@sustainable.test / demo123
```

4. Unit Tests (xUnit)

```
cd tests/SustainablePS.Tests
dotnet test
# All 15 tests should pass
```

Shared database: All three applications read and write the same sustainableps.db SQLite file.

To share data between the API and Blazor web, set the environment variable:

```
SUSTAINABLEPS_DATA_PATH=/absolute/path/to/sustainableps.db
```

Each video is at least 2 minutes and covers the desktop merchant console, the customer web application, and the REST API.

10. Conclusion

The Sustainable Product Shopping Platform v2 delivers a complete .NET 9 marketplace application demonstrating the full range of SE-410 learning objectives:

- Database (SQLite, EF Core): Relative-path connection string; schema defined via Fluent API; EF transactions for atomic checkout; EnsureCreated for zero-config startup.
- ASP.NET Web API: Minimal API with 16 REST endpoints; CORS; OpenAPI/Swagger; DTO records; full JSON responses.
- LINQ: Seven distinct queries including GroupBy, SelectMany, Include, Join, Average, Sum, OrderBy, FirstOrDefault — all demonstrated with explanations.
- Delegates and Events: Four custom delegate types (OrderConfirmedHandler, StockChangedHandler, OrderStatusChangedHandler, ProductAddedHandler) wired as events and verified by xUnit tests.
- Desktop (MAUI): Merchant console with product management, stock control, order fulfillment, and notification centre.
- Web (Blazor Server): Customer shopping experience with search, cart, checkout, carbon dashboard, order history, and notifications.
- Unit Tests (xUnit): 15 test cases; InMemory EF Core provider for isolation; 100% pass rate; covers authentication, events, cart, checkout, analytics.
- Comments: XML documentation on all public types, methods, and properties; inline comments explain every design decision.

The platform demonstrates how a single SustainablePS.Core business layer can simultaneously power a desktop application, a web application, and a REST API — all sharing one database — while keeping concerns cleanly separated and fully testable.